Technology
Arts Sciences
TH Köln

Entwicklungsprojekt interaktiver Systeme

Dokumentation MS3

Christopher Huntscha, Sören Lehmann

20.01.2019

Inhaltsverzeichnis

1. Einleitung 3
    2. Abweichungen 4
        2.1 Systemarchitektur 4
        2.2 Screendesign 6
    3. Implementierung 8
        3.1 Server 8
        4.1 Server 9
        4.2 Client 9
    5. Prozessassessment 10
        5.1 Erster Meilenstein 10
        5.2 Zweiter Meilenstein 10
        5.3 Dritter Meilenstein 11
    6. Fazit 12
    7. Quellenverzeichnis 13

1. Einleitung

In diesem Dokument werden die Ergebnisse des Projektverlaufs dokumentiert und aus der Sicht des Entwicklungsteams bewertet. Die folgenden Abschnitte werden von der Ausarbeitung des zweiten Meilensteins geprägt.

Während der Bearbeitung sind jedoch Differenzen aufgefallen, die so nicht im zweiten Meilenstein geplant waren. Folglich werden zuerst diese Unterschiede erklärt.

2. Abweichungen

Zwar wurde der präskriptive Stand des Projekts mehrmals iterativ erarbeitet, jedoch hat sich die Schwierigkeit gezeigt den eigenen Zeitplan einzuhalten. Somit wurden Prioritäten gesetzt und nur Teilaspekte für den Prototypen implementiert.

2.1 Systemarchitektur

Das Architekturmodell wurde folglich so umgestaltet, dass der Arbeitsaufwand des Projektteams nicht zu hoch für die Umsetzung ausfällt.

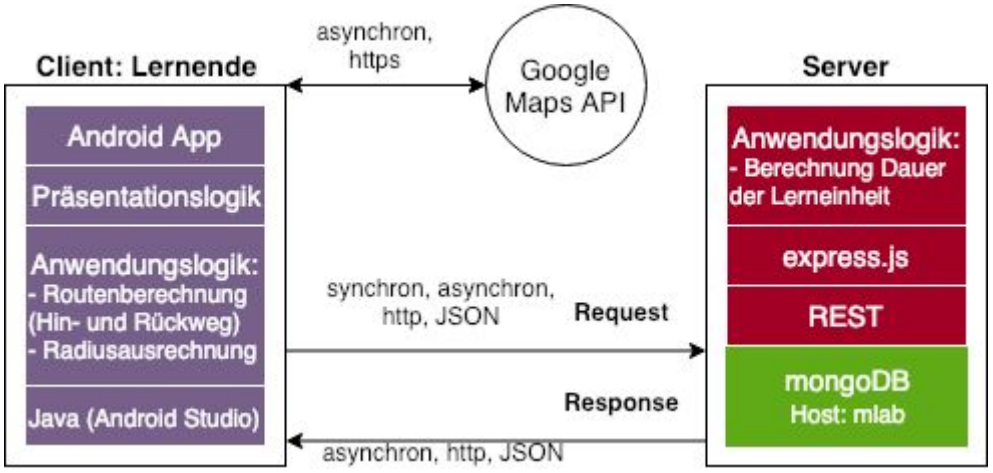

Abbildung 1: Systemarchitektur Prototyp

In den folgenden Abschnitten werden die Gründe für die Reduzierung des Modells erläutert, in Bezug auf die zuvor erarbeitete Systemarchitektur (*siehe MS2*).

2.1.1 Client: Lehrender

Für den Client des Lehrenden wurde im Projektverlauf keine Anwendungslogik erarbeitet. Da der Prototyp das Alleinstellungsmerkmal durch die Anwendungslogik veranschaulichen soll, wurde sich dazu entschieden diesen Client nicht weiter zu berücksichtigen.

2.1.2 Datenspeicher

Um die Nutzung des Anwendungssystems auch offline zu gewährleisten, war eine Speicherung der Karteikarten und Karteikartenstapel geplant. Die Priorität für den Prototypen ist jedoch nicht hoch genug, um mit diesem Aspekt die Funktionalität zu zeigen. Eine Implementierung eines Datenspeichers für den Client ist somit weiterhin nicht beachtet wurden.

2.1.3 Anbindung: Server - Google Maps API

In den Modellierungsbegründungen des zweiten Meilensteins wurde ein Datenaustausch zwischen Server und Google Maps API dargestellt, um Markierungen auf der Google Maps

API bereits in der Datenbank ablegen zu können. Dieser Aspekt wurde jedoch missverstanden und bei der Implementierung des Prototyps rausgelassen. Die Markierungen der Karte sind ausschließlich von der Position des Clients abhängig und können somit nicht zuvor von dem Server ermittelt werden, ohne die GPS-Daten des Nutzers zu haben.

2.1.4 Anwendungslogik: Filterungen

Da Zeit eine wichtige Rolle für die Fertigstellung des Projekts spielt, wurden weitere Prioritäten für die Anwendungslogiken gesetzt. Filterungsmöglichkeiten können für eine umfassendere Anwendungslogik des Projekts sorgen, beweisen jedoch keine Alleinstellungsmerkmale. Das Filterungssystem der Karteikarten nach Fehlern ähnelt dem von Phase 6 und die Filterung nach eigenen Präferenzen wird in mehreren Anwendungssystemen, wie AnkiDroid, vertreten.

2.2 Screendesign

Das Screendesign wurde größtenteils aus dem Projekt Konzept übernommen. Jedoch mussten ein paar Änderungen vorgenommen werden, damit Menüs an Android angezeigt werden können. Dies sticht am meisten im Menü (siehe Abbildung 3) heraus. Hinzu kommt auch noch die Einhaltung des Material Designs, welches als Designprinzip für neue Applikationen von Google vorgeschlagen und unterstützt wird.

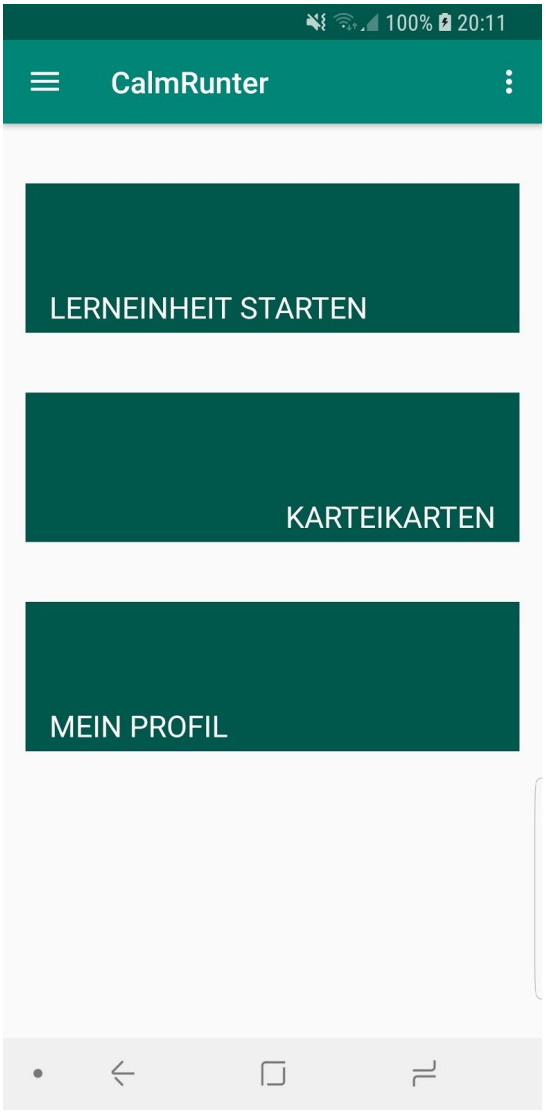

Abbildung 2: Homescreen

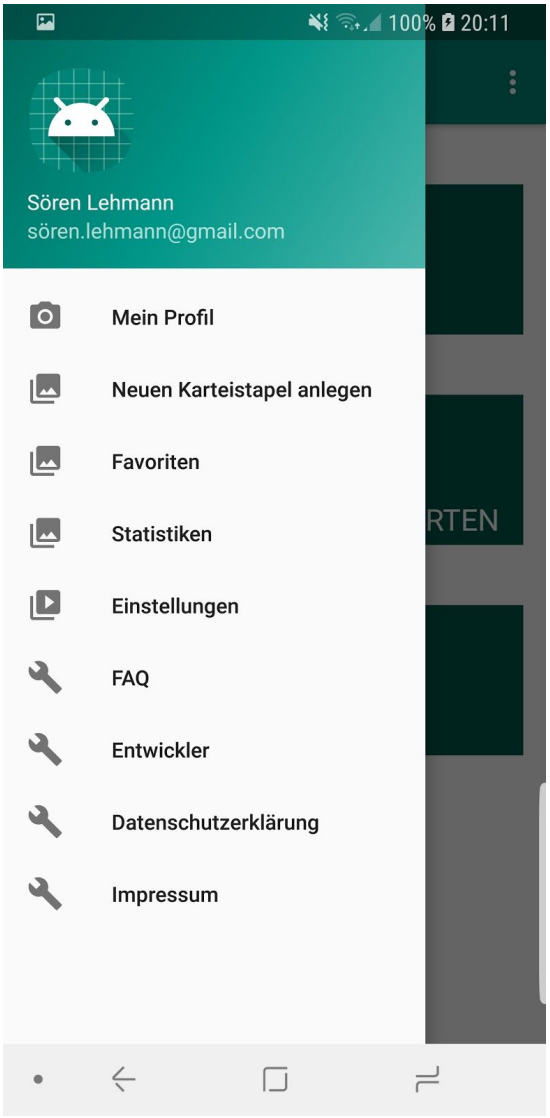

Abbildung 3: Menü

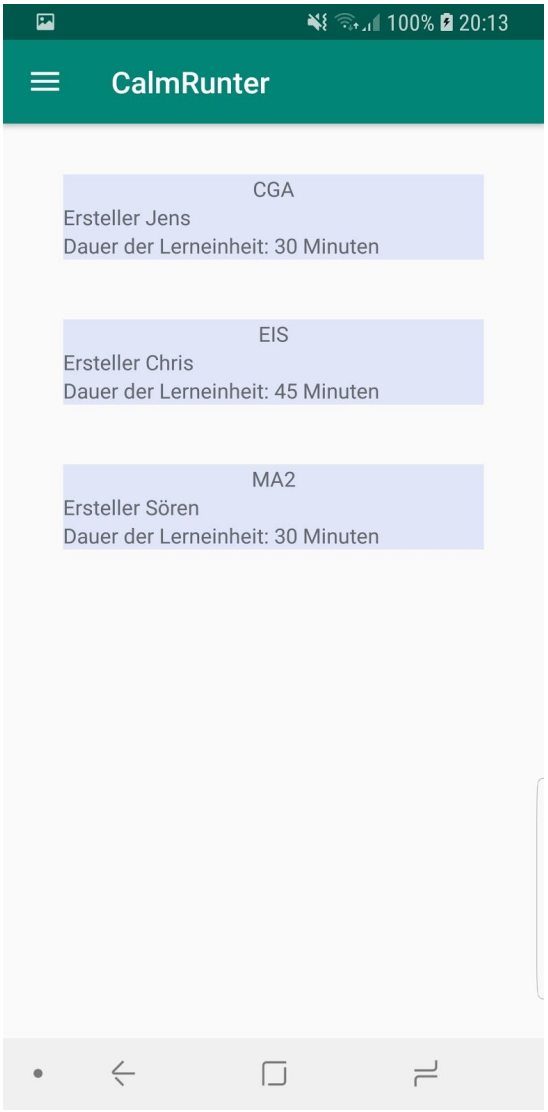

Abbildung 4: Auswahl Karteikartenstapel

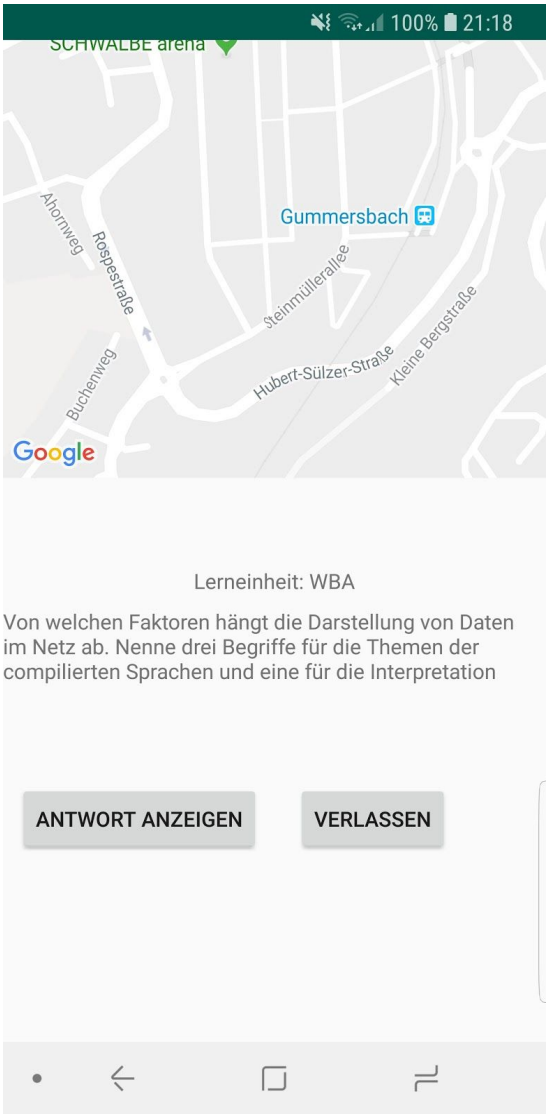

Abbildung 5: Gestartete Lerneinheit mit Frage

Zur besseren Lesbarkeit und Anklickbarkeit wurde entschieden, dass die Buttons der Karteikarten recht groß gehalten werden. Die wichtigsten Attributen beschreiben die einzelnen Karteikartenstapel, wie der Name, Ersteller und die Dauer der Lerneinheit (siehe Abbildung 4).

Im Screen der gestarteten Lerneinheit wurde das Designprinzip aus dem Konzept weiter ausgebaut. Der Screen ist unterteilt in zwei verschiedene Bereiche: Der Obere zeigt die Route an, die gegangen werden kann, und unterhalb dieser Karte werden die Karteikarten durchlaufen.

3. Implementierung

Die folgenden Abschnitten erläutern die Ergebnisse und Absichten der Implementierung.

3.1 Server

In Bezug auf den Server wurde sich an einem Video Tutorial[1] orientiert, um den Einstieg in die Implementation zu vereinfachen.
Der Server wurde, wie bereits in den Abweichungen beschrieben, für den Prototyp ausreichend umgesetzt. Die *server.js* Datei dient hierbei zur Bereitstellung der Module, Anbindung von MongoDB und Initialisierung der Routen.
Im Ordner *routes* befinden sich Java Script Dokumente, die den Umgang mit requests ermöglichen. Jede Instanz bietet hierbei eine Ausgabe aller vorhandenen Objekte, eine Ausgabe spezieller Objekte, das Erstellen eines Objekts, das Löschen eines Objekts und das Ändern eines Objekts. Zwar ist die Umsetzung nicht für jedes Element nötig, schadet aber nicht in der Anbindung von Server und Client. Zu Beginn wird ein Datenschema importiert, welches den Umgang mit den Objekten ermöglicht.
Im Ordner *models* werden diese Datenstrukturen festgelegt und mit wichtigen Eigenschaften, wie *unique*, versehen. Unique bietet hierbei ein Erkennungsmerkmal von Primärschlüsseln, die ein Objekt vollständig abhängig von seiner Identifikation macht. Die Datenstrukturen wurden in einem anderen Ordner ausgelegt, um eine bessere Übersicht von dem Code zu gewährleisten.
Für die Anwendungslogik des Servers wurde eine Algorithmus im Pfad *server/routes/karteikartenstapel.js* implementiert, der die Zeitdauer eines Karteikartenstapels immer dann aktualisiert, wenn eine PUT-Request erfolgreich verschickt wurde. Hierzu werden die Zeitdauer-Werte der abgelegten Karteikarten addiert und abgespeichert.

3.2 Client

Für den Clienten wurde eine Anbindung an den Server umgesetzt, welche aber zum Zeitpunkt der Abgabe nicht im Code aktiviert wurde. Die Datensätze wurden lokal auf dem Clienten erzeugt, um mehr Fokus auf die Anwendungslogik setzen zu können. Damit Ressourcen einer URI gelesen werden können, musste eine Klasse *HttpHandler* angelegt werden, welche die URI auf fehlende Eigenschaften untersucht und falls nötig Fehlermeldungen ausgibt, sowie die Klasse *ParsingJSONinActivity*, die das Parsen der Ressourcen ermöglicht.
Momentan wird auch die Radiusberechnung eines Karteikartenstapeln durch Anwendung eines Algorithmuses lokal berechnet, welcher mit in die Routenberechnung einfließt.
Die eigentliche Routenberechnung stellt aber eine herausfordernde Implementation dar. Innerhalb des Bearbeitungszeitraums wurde kein Lösungsansatz für die Anwendungslogik gefunden, um einen Rundkurs über eine gewissen Distanz oder einen Radius berechnen zu lassen. Somit muss an der Radiuslinie ein zufälliger Punkt auf einer Straße gesetzt werden, der für den Hinweg als Endpunkt und für den Rückweg als Startpunkt gilt. Dies konnte aus Zeitgründen noch nicht in vollem Umfang umgesetzt werden.

4. Installationsdokumentation

Für das Anwendungssystem wurde sich für eine MongoDB entschieden, die über https://mlab.com gehostet wird. Die Anmeldedaten sind in der *server.js* Datei zu finden.

1. Repository herunterladen (*https://github.com/soeleh/EISWS1819HuntschaLehmann.git*)
2. node.js installieren (*https://nodejs.org/en/download/*)

4.1 Server

1. Das Verzeichnis MS3/Prototyp/server öffnen
2. Das Terminal/Die Konsole in diesem Verzeichnis öffnen
3. Den Befehl *npm install* ausführen (dies kann einige Minuten dauern)
4. Mit dem Befehl *node server.js* den Server starten
5. Mit einem beliebigen REST Client Requests an *localhost:2019/* senden

4.2 Client

1. Die CalmRunter.apk aus dem Ordner EISWS1819HuntschaLehmann/MS3 auf das Android Endgerät laden und installieren, mit Android Studios eigenen Emulator verwenden oder über Android Studios die Applikation direkt auf das Endgerät aufspielen
    - Hier ist darauf zu achten, dass die virtuelle Umgebung den Google PlayStore Service nicht besitzt, da sonst das Anzeigen der Map nicht funktioniert
2. Im App Menü taucht nun die CalmRunter Applikation auf und kann ausgeführt werden.
3. Es müssen eventuell verschiedene Rechte, wie Internetfreigabe, GPS Lokalisierung der Applikation gegeben werden.

5. Prozessassessment

In dem folgenden Abschnitt wird eine kritische Reflexion des Projektverlaufs, hinsichtlich der Meilensteine und dem methodischen Rahmen, beschrieben.

5.1 Erster Meilenstein

Für diesen Meilenstein musste ein Konzept und Rapid Prototype fertiggestellt werden.
Das Konzept wurde durch die Maßgaben der Workshops ausgearbeitet. Vor der Fertigstellung hat das Team den Betreuern eine inhaltliche Übersicht veranschaulicht, welche als Hilfe zum Verständnis dienen sollte. Jedoch differenzierten sich die Vorstellungen des Teams über den Inhalt mit eigentlichen Bewertung. Das hätte vorher intensiver klargestellt werden müssen.
Auch der Rapid Prototype wurde projektspezifisch ausgewählt und veranschaulichte eine erste Bedienung der Google Maps API anhand einer geringen Anwendungslogik. Da bereits viel Zeit für die Fertigstellung des Exposés verwendet wurde, blieb nicht viel mehr Zeit für einen umfassenderen Rapid Prototype, wodurch das Ergebnis mager ausfiel.

5.2 Zweiter Meilenstein

Der zweite Meilenstein verlief deutlich strukturierter und umfassender.
Zu Beginn wurde ein Zeitplan im Projektplan aufgestellt, der alle Artefakte mit Unterkapiteln auflistete und den Entwicklern zugeteilt wurde. Hier fiel jedoch im weiteren Verlauf auf, dass die Planung nicht wie gewünscht verlief.
Von großer Hilfe war jedoch die Kritik der Betreuer und Professoren an den Ergebnissen des ersten Meilensteins und das Ausleihen des Buchs “The Usability Engineering Lifecycle”von Deborah Mayhew. Während der Bearbeitung wurde sich nicht nur an diesen Punkten orientiert, sondern auch mit Kommilitonen ausgetauscht. Die Perspektive von aussenstehenden Fachleuten hat sich als sehr hilfreich erwiesen, wodurch das Ergebnis der Ausarbeitung sehr zufriedenstellend ausfiel.

5.3 Dritter Meilenstein

Um einen präsentierfähigen Prototypen fertigzustellen war zu Beginn klar, dass nicht alle Erarbeitungen berücksichtigt werden können, da hierfür die Zeit fehlte. Beispielsweise wurden nicht alle Statuscodes und Ergebnisse der Nutzeranalysen für den Prototyp berücksichtigt.
Nur ein Teammitglied hatte bislang Erfahrung mit der Implementation einer Android App in einem Wahlpflichtfach gesammelt. Somit wurde der Arbeitsaufwand beider Teammitglieder mit der Implementation von Server und Client aufgeteilt. Es musste eine Priorisierung der Funktionen des Prototyps vorgenommen werden, um eine sinnvolle Verteiltheit zu gewährleisten.
Zur Orientierung verhalfen die Ergebnisse des zweiten Meilenstein, die sowohl das Screendesign, als auch die Implementation der Anwendungslogiken stark beeinflussten. Jedoch wurde die geplante Anwendungslogik des Clients nicht vollständig umgesetzt. Dies ist einer fehlinterpretieren Zeitplanung zu verschulden, die zukünftig genauer thematisiert werden sollte.

6. Fazit

Zusammenfassend ist die Zielsetzung für das Entwicklerteam ausreichend erfüllt worden, hätte jedoch zufriedenstellender sein können, wenn mehr Zeit gegeben wär. Da das Projekt auf eine Verteiltheit der Anwendungslogik abzielt, musste ein großer Zeitverlust durch das Einarbeiten in Android Studio hingenommen werden.
Mit mehr Erfahrung in der Programmierung mit Java könnten mehrere Komponenten der Systemarchitektur des zweiten Meilensteins implementiert werden. Auch das Screendesign hätte der Ausarbeitung des konzeptuellen Modells deutlicher geähnelt.

7. Quellenverzeichnis

1 https://www.youtube.com/watch?v=o3ka5fYysBM (Stand: 20.01.2019 22:12)